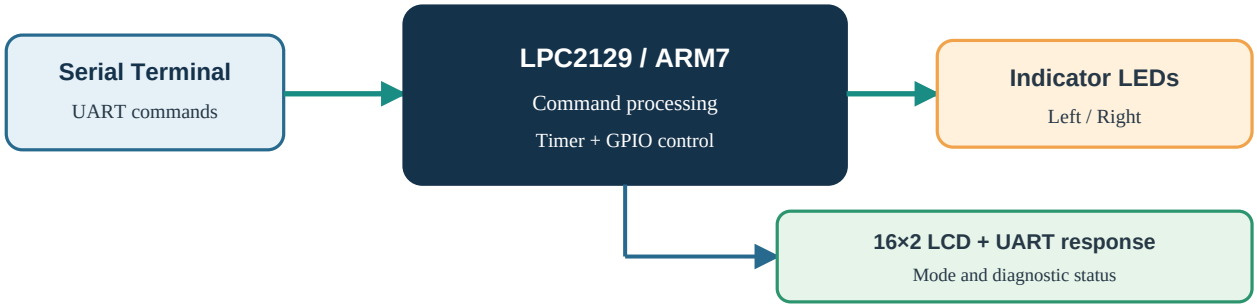
SMART AUTOMOTIVE INDICATOR CONTROLLER

Project Report and Technical Explanation

LPC2129 / ARM7 • Embedded C • UART • GPIO • LCD • Timer 0



Project purpose
To design and simulate a microcontroller-based indicator controller that accepts commands over UART, controls left/right/hazard indicator patterns, displays the selected mode on an LCD, and returns status information through the serial terminal.

Prepared as a technical explanation of the project design, operation, firmware logic, and testing approach.

Note: This is an educational simulation prototype. It is not production automotive safety firmware and must not be connected to a real vehicle.

Contents

• 1. Abstract

• 2. Introduction

• 3. Problem statement

• 4. Project objectives

• 5. System overview

• 6. Hardware and software requirements

• 7. Hardware connections and circuit description

• 8. UART command protocol

• 9. Working principle

• 10. Firmware design and program flow

• 11. Timer-based indicator operation

• 12. LCD and serial diagnostics

• 13. Testing and expected results

• 14. Limitations

• 15. Conclusion and future enhancements

Report focus

This document explains the project as if presenting it to a technical reviewer: what the system does, why each module is used, how data moves through the system, how commands change the outputs, and how the design should be tested. It does not treat repository management or social-media posting as project chapters.

1. Abstract

This project presents the design of a Smart Automotive Indicator Controller using an LPC2129 ARM7 microcontroller. The system accepts user commands through UART communication and selects one of four operating modes: left indicator, right indicator, hazard warning, or all indicators off. A 16×2 LCD displays the current mode, while the serial terminal provides command feedback and diagnostic status. Timer 0 is intended to generate periodic timing events for flashing the indicator LEDs. The design combines UART, GPIO, LCD interfacing, timer configuration, command validation, and state-machine-based control in a single embedded application. The system is developed and tested in a simulation environment before any optional hardware implementation. The project demonstrates modular firmware design and basic automotive body-control concepts; it does not claim to meet production automotive safety standards.

2. Introduction

Automotive electronic control units use embedded software to read commands or sensor inputs, make decisions, and control electrical outputs. Indicator control is a useful learning example because it involves distinct operating modes, periodic output changes, and clear conditions for switching between modes. In this project, the LPC2129 acts as the controller. UART provides a simple command interface, GPIO drives the indicator LEDs, Timer 0 provides the timing reference, and the LCD presents local status information.

The project is intentionally organized into separate driver and application modules. UART and LCD functions handle low-level communication, while the application layer decides which indicator mode should be active. This separation makes the program easier to debug, extend, and explain during a project review.

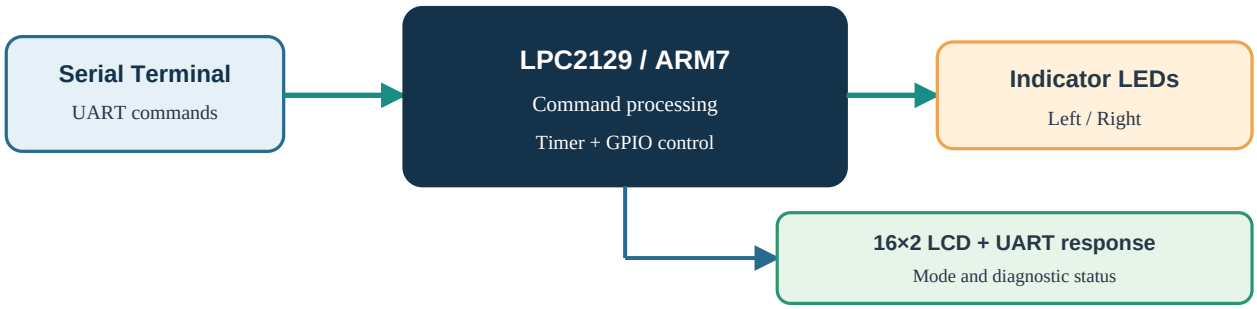
3. Problem statement

A basic LED demonstration usually controls an output directly and does not show how an embedded application manages several operating modes. The objective here is to create a small controller that accepts commands, selects a valid mode, updates multiple outputs consistently, and provides visible feedback. The software must avoid leaving an unwanted indicator active when the operating mode changes, and it should continue accepting commands while the indicators flash.

4. Project objectives

- Interface the LPC2129 with a serial terminal using UART0.
- Control left and right indicator LEDs using GPIO outputs.
- Implement OFF, LEFT, RIGHT, and HAZARD operating modes.
- Use Timer 0 to schedule indicator flashing.
- Display the selected mode on a 16×2 LCD.
- Report status and invalid commands through UART.
- Test the command behavior and mode transitions in Proteus.

5. System overview



The system has three main paths. The input path carries a command from the serial terminal to UART0. The control path interprets the command and updates the selected operating mode. The output path drives the indicator LEDs and refreshes the LCD and serial response. Timer 0 provides periodic events for blinking modes.

Main modules

Module	Role in the project
UART0 driver	Initializes serial communication, receives command bytes, and transmits messages.
Command processor	Validates the input command and selects the operating mode.
GPIO control	Turns left/right indicator outputs on or off according to the mode and blink phase.
Timer 0	Provides a periodic timing event used to toggle indicator outputs.
LCD driver	Displays mode and status messages on the 16×2 LCD.

6. Hardware and software requirements

Item	Purpose
LPC2129 ARM7 microcontroller	Runs the firmware and controls peripherals.
16×2 LCD	Shows the active indicator mode and status.
Two or more LEDs with resistors	Represent left and right indicator lamps in simulation.
Virtual Terminal	Sends commands and displays UART responses.
Keil µVision	Compiles the Embedded C source and generates the HEX file.
Proteus	Simulates the controller, serial terminal, display, and LEDs.

7. Hardware connections and circuit description

The Proteus schematic should connect the microcontroller UART0 pins to the Virtual Terminal using crossed transmit and receive lines. The MCU TXD0 signal connects to terminal RXD; terminal TXD connects to MCU RXD0. Configure both ends for the same baud rate and frame format. The LCD must be wired according to the pin masks used by the existing LCD driver, and each indicator LED must be connected to a GPIO pin through a current-limiting resistor in any physical circuit.

Signal	Connection	Purpose / check
TXD0	MCU TXD0 → terminal RXD	Displays firmware messages on the terminal.
RXD0	Terminal TXD → MCU RXD0	Carries user commands to the MCU.
Left LED	GPIO output → resistor → LED	Represents left indicator state.
Right LED	GPIO output → resistor → LED	Represents right indicator state.
LCD	RS, EN, D4–D7 per LCD driver	Displays mode and status.

Clock and baud-rate dependency

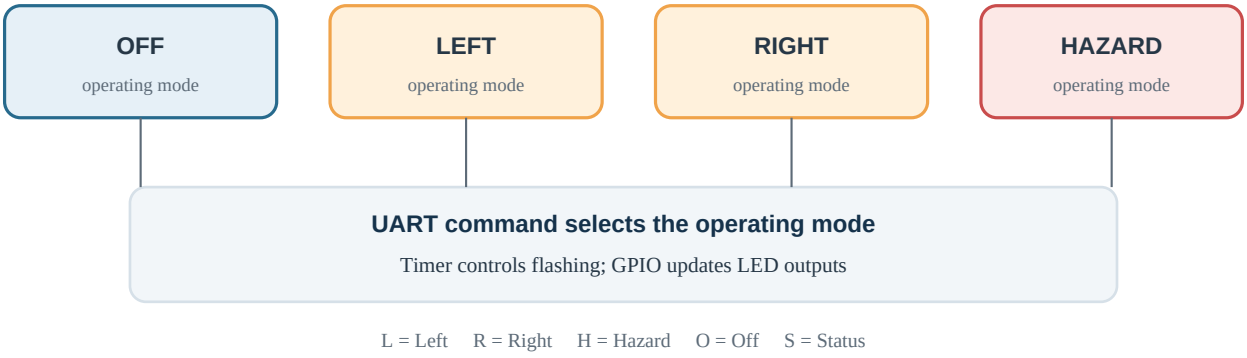
The UART baud-rate divisor depends on the peripheral clock (PCLK). The firmware clock assumptions must match the clock configured in the Proteus microcontroller properties and the VPBDIV setting. If PCLK is 15 MHz and the requested baud is 9600, the divisor is approximately 97 using the common 16× baud divisor formula. A mismatch can cause a blank or unreadable terminal even when the program compiles.

8. UART command protocol

Command	Meaning	Expected output behavior
L	Left indicator	Left LED flashes; right LED remains off.
R	Right indicator	Right LED flashes; left LED remains off.
H	Hazard mode	Left and right LEDs flash together.
O	Off	Both LEDs switch off immediately.
S	Status request	Reports the current mode without changing it.
Other	Invalid command	Sends an error response and keeps outputs in a defined state.

9. Working principle

At startup, the firmware initializes UART0, the LCD, GPIO pins, and Timer 0. It sets the default mode to OFF and displays a ready message. The main loop checks for incoming commands. When a valid command arrives, the command processor updates the operating mode. The output-control function applies the correct LED pattern, while the LCD and UART report the new state.



Mode behavior

Mode	Left LED	Right LED	Description
OFF	OFF	OFF	All indicators are disabled.
LEFT	Flashing	OFF	Only the left indicator flashes.
RIGHT	OFF	Flashing	Only the right indicator flashes.
HAZARD	Flashing	Flashing	Both indicators flash together.

10. Firmware design and program flow

The firmware should be divided into small functions. uart0_init() configures UART0; uart0_rx() obtains a command byte; process_command() validates the byte; set_mode() changes the state; indicator_tick() updates the LED pattern on each timer event; lcd_show_mode() displays the mode; and uart_report_status() sends diagnostic text.

Program flow

Step	Operation
1	Initialize UART0, LCD, GPIO, and Timer 0.
2	Set mode to OFF and clear the indicator outputs.
3	Receive a command byte from UART0.
4	Validate the command and select the operating mode.
5	Apply output state and update LCD/UART feedback.
6	On each timer event, toggle only the outputs required by the current mode.
7	Repeat to accept the next command.

11. Timer-based indicator operation

A vehicle indicator must flash at a steady rate. Timer 0 can generate a periodic event that changes the LED output state. For example, a 250 ms timer event can toggle the output every 250 ms, resulting in an ON interval of 250 ms and an OFF interval of 250 ms (a complete cycle of 500 ms). The exact period should be chosen for demonstration and is not a claim of regulatory compliance.

A non-blocking design is preferred. A long delay inside the main loop prevents the program from checking UART frequently. Instead, the timer event should update a tick flag or invoke a short timer handler, and the main application should process the event while continuing to receive commands.

Clock calculation concept

```
timer interval depends on PCLK, prescaler, match value, and timer reset configuration
```

Configure the prescaler and match register according to the LPC2129 user manual. Verify whether the timer counter increments every PCLK cycle or after prescaling, and configure the match control register to reset or interrupt as intended.

12. LCD and serial diagnostics

The LCD gives immediate local feedback even when a terminal is not visible. A possible display layout is shown below.

Example serial response

```
SMART INDICATOR CONTROLLER READY\r\nMODE=OFF\r\nCOMMAND=H\r\nMODE=HAZARD\r\n
```

Diagnostics can include current mode, total valid commands, invalid command count, and software fault flags. A software flag is only a simulated fault unless real electrical sensing hardware is added.

13. Testing and expected results

Test	Input/action	Expected result
1	Start simulation	LCD shows OFF/ready; terminal receives startup message.
2	Send L	Left LED flashes; right LED stays off.
3	Send R	Right LED flashes; left LED stays off.
4	Send H	Both LEDs flash together.
5	Send O	Both LEDs turn off immediately.
6	Send S	Current mode is reported; mode remains unchanged.
7	Send invalid character X	Error message appears; active mode remains defined.
8	Change modes repeatedly	No stale LED remains on after mode changes.

14. Limitations

• The current design is a simulation-oriented learning project; it has not been validated for use in a real vehicle.

• The firmware depends on the exact CPU clock, PCLK, pin mux, timer configuration, and Proteus device model.

• LEDs represent indicator lamps. The project does not measure real lamp current or detect an open/short circuit unless sensing circuitry is added.

• A blocking UART receive function can stop the main loop while waiting for input. A non-blocking receiver or UART interrupt/ring buffer is preferable for a responsive timer-based system.

• Real automotive development requires additional design, diagnostics, electrical protection, testing, and safety processes beyond this prototype.

15. Conclusion and future enhancements

The Smart Automotive Indicator Controller demonstrates how a microcontroller can combine serial communication, GPIO control, LCD interfacing, and timer-based behavior in one application. The UART command interface allows the user to select left, right, hazard, and off modes, while the LCD and serial terminal provide feedback. A state-machine-based design helps keep mode transitions clear and prevents conflicting outputs.

The project can be extended by adding UART receive interrupts and a ring buffer, debounced physical switches, real input sensing, a CAN communication interface where supported by the chosen target, watchdog handling, and more extensive automated tests. Each extension should be implemented and verified separately.

Summary for a project presentation

“I designed a simulation-based automotive indicator controller using the LPC2129 ARM7 microcontroller. The controller receives commands through UART, selects left, right, hazard, or off mode, and controls indicator LEDs through GPIO. Timer 0 provides the periodic timing for flashing, while a 16×2 LCD and UART messages show the system status. The project helped me integrate peripheral drivers, implement command handling and state-based control, and test embedded firmware in Proteus.”

End of report